

Lab Work 06

Topic	try-except-else-finally, Raising exceptions, Custom exceptions
Objective	Implement robust error handling using Python's exception mechanisms and create custom exception types.
AI Copilot Usage	Not Allowed

Tasks

1. Basic *try-except*:

- In the ***PersonFactory.create_person*** method, the current code prints errors but returns ***None***. Modify it to wrap the instantiation logic (e.g., ***Student(...)***, ***Teacher(...)***) inside a ***try...except KeyError*** block to handle missing ***kwargs***.
- Also add ***except ValueError*** to catch the ID validation errors raised by the constructors.
- Inside the ***except*** blocks, print a more informative error message and *re-raise* a custom exception (see Task 2) or return ***None*** (discuss the trade-offs). For now, let's just print and return ***None***.
- Test by calling the factory with missing or invalid data.

2. Custom Exceptions:

- Define custom exception classes that inherit from ***Exception***:
 - SchoolSystemError*** (as a base custom exception)
 - EnrollmentError(SchoolSystemError)***
 - FactoryError(SchoolSystemError)***
 - InvalidDataError(SchoolSystemError)***
- Modify the ***Unit.add_student*** method: Instead of just printing errors (like "Unit is full" or "Cannot enroll non-student"), ***raise*** an ***EnrollmentError*** with an appropriate message.
- Modify the ***PersonFactory.create_person***: Instead of returning ***None*** on error (from Task 1), ***raise*** a ***FactoryError*** or ***InvalidDataError*** wrapping the original error message if helpful (exception chaining - see Creative Challenge).

3. Handling Custom Exceptions:

- Modify the ***register_student_for_unit*** function. Wrap the call to ***unit.add_student(student)*** in a ***try...except EnrollmentError as e:*** block. Print the specific error message from the caught exception ***e***.

- b) Modify the main testing block (*if __name__ == "__main__":*). When calling *factory.create_person*, wrap it in a *try...except (FactoryError, InvalidDataError) as e:* block to catch the errors raised by the factory. Print the error.
4. **finally Clause:**
- a) Imagine loading student data from a file (simulate this). Write a block of code that opens a (mock) file, reads data (can just be *print("Reading data...")*), processes it (can lead to an error, e.g., *raise ValueError("Bad data")*), and uses a *finally* block to ensure a "Closing file..." message is printed, regardless of whether an error occurred during processing.
5. **Creative Challenge (Optional, if time permits):**
- a) Implement **exception chaining**. In the *PersonFactory*, when you catch a *KeyError* or *ValueError*, raise your *FactoryError* like this: *raise FactoryError(f"Failed to create person: {e}") from e*.
- b) Modify the *except* block in your main test code to catch *FactoryError* and print both the *FactoryError* message (*e*) and its cause (*e.__cause__*) to see the original error.

Important Notes

- This is an individual task. All code must be your own original work.
- Ask questions and seek help from the lecturer or teaching assistants if you are stuck.
- Focus on writing clean, well-organized, and well-documented code.